M1 MIAGE · PROJET INTÉGRATEUR · GROUPE 3

FestiBed

Réserver ensemble des Pass de festival et des hébergements,
sans jamais vendre deux fois la même place

Présentation du 23/10/2026 · Oracle · Kotlin Spring · JPA · eCloud

COMPRÉHENSION DU SUJET

Trois règles structurent FestiBed

1

Réservation groupée

Pas de Pass sans hébergement, pas d’hébergement sans Pass. La capacité des hébergements couvre le nombre de Pass.

2

Panier temporaire

Le panier expire après 15 minutes. Y mettre un élément ne constitue pas une réservation.

3

Validation transactionnelle

Les Pass et les nuitées sont réservés ensemble, ou rien ne l’est. C’est le cœur du projet (ACID).

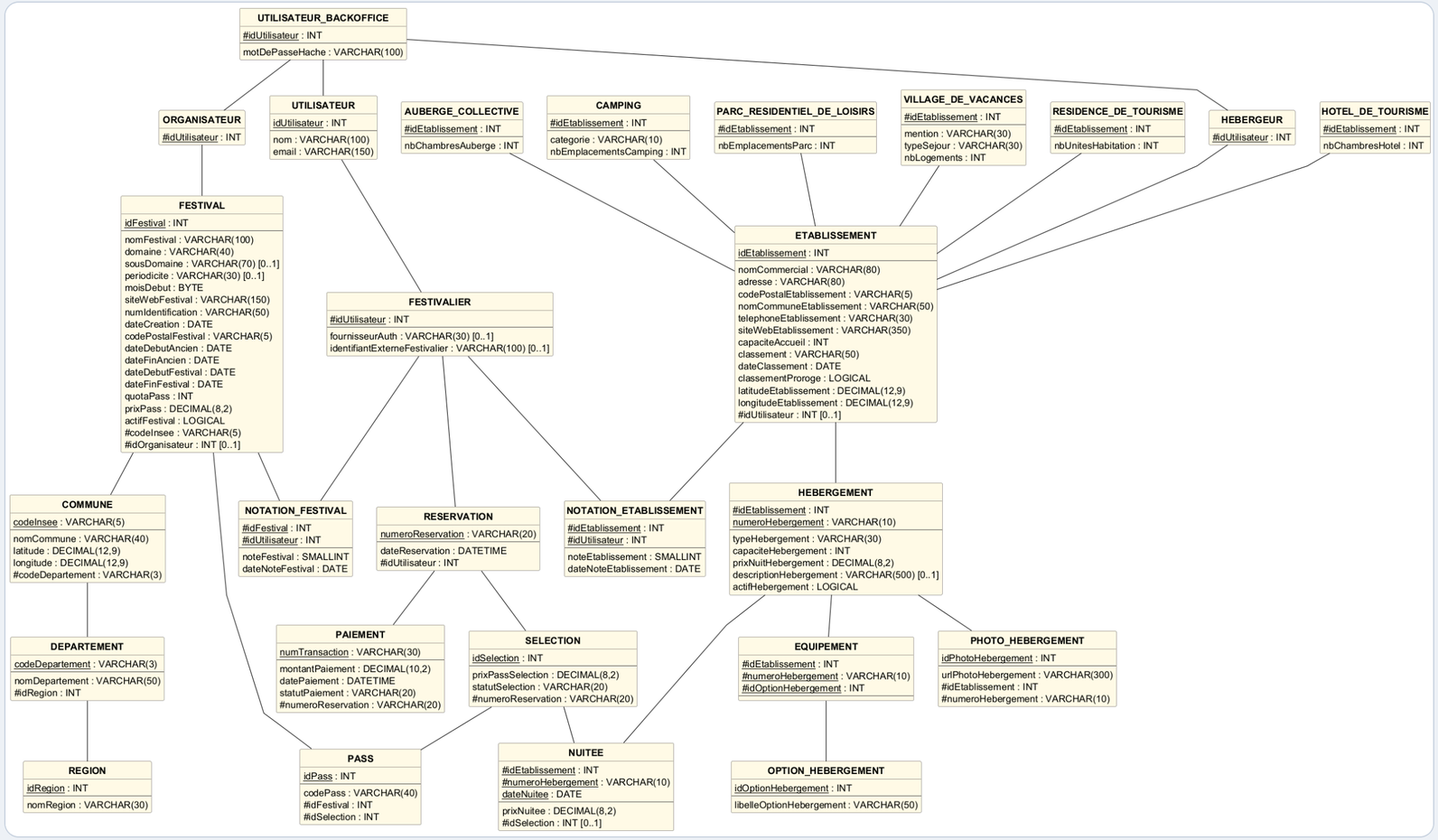
NOS CHOIX ARGUMENTÉS

Chaque choix répond à une contrainte

Choix	Pourquoi
Panier dans l'IHM, rien en base avant la validation	Aucun verrou pendant que le festivalier réfléchit
Nuitée identifiée par (hébergement, date)	La clé primaire interdit deux réservations de la même nuit
Pass rattaché au Festival	On sait combien de places sont vendues par festival
Classement, périodicité, domaines en **énumérations**	Peu de valeurs fixes : moins de tables et de jointures
Héritage **JOINED** (Utilisateur, Établissement)	Les sous-types ont leurs propres attributs
Authentification à la validation seulement	Le sujet ne l'exige qu'au moment de valider

MODÈLE DE DONNÉES

MLD : 27 tables, quatre blocs



Utilisateurs
Festivalier, Organisateur, Hébergeur

Offre
Festival, Établissement, Hébergement

Stock
Nuitée, Pass

Vente
Réservation, Sélection, Paiement

MODÈLE DE DONNÉES

Le modèle protège déjà contre la survente

Nuitée = (hébergement, date)

`PK (idEtablissement, numeroHebergement, dateNuitee)`

Une même nuit d’une chambre n’existe qu’une fois : c’est la base qui le garantit, pas le code.

Occupation 0..1

`Nuitee.idSelection NULL = libre`

Une nuitée appartient au plus à une sélection. Réserver = passer de NULL à une valeur.

Pass → Festival

`quotaPass + nbPassVendus`

Chaque Pass est une place d’un festival. Un compteur rend le contrôle du quota immédiat (ajout proposé).

MODÈLE DE PROCESSUS

La base n’est touchée qu’à la validation

IHM · 15 min max

Recherche et panier

Festival, Pass, un ou plusieurs hébergements

→

Réseau social

Authentification

Seulement au moment de valider

→

Serveur · 1 transaction

Validation

Vérifie et réserve Pass + nuitées

✓ **Succès** : réservation créée, Pass et bon d’hébergement en PDF

! **Échec** : rien n’est réservé, « validation échouée »

GESTION DES TRANSACTIONS

Vérifier et réserver en une seule instruction

```
BEGIN -- méthode @Transactional, READ COMMITTED
UPDATE Nuitee SET idSelection = :s
 WHERE (idEtablissement, numeroHebergement) = (:e, :h)
  AND dateNuitee IN (:nuits) AND idSelection IS NULL;
-- lignes modifiées < nuits demandées → ROLLBACK
UPDATE Festival SET nbPassVendus = nbPassVendus + :n
 WHERE idFestival = :f AND nbPassVendus + :n <= quotaPass;
-- 0 ligne modifiée → ROLLBACK : plus de Pass
INSERT Reservation, Selection, Pass …
COMMIT;
```

Pas de « SELECT puis UPDATE » : la condition et l'écriture sont atomiques, aucune fenêtre entre « je vois une place » et « je la prends ».

MISE EN ŒUVRE · SPRING BOOT + KOTLIN

Spring gère le COMMIT et le ROLLBACK pour nous

```
@Transactional(isolation = READ_COMMITTED, timeout = 5)
fun valider(panier: Panier, client: Festivalier): Reservation {
  val resa = reservationRepo.save(Reservation(client))
  for (s in panier.selections.sortedBy { it.idFestival }) {
    val sel = selectionRepo.save(Selection(resa, s))
    if (nuiteeRepo.occuper(sel, s.hebergement, s.nuits) < s.nuits.size)
      throw IndisponibleException("nuitée") // → ROLLBACK
    if (festivalRepo.reserverPass(s.idFestival, s.nbPass) == 0)
      throw IndisponibleException("pass") // → ROLLBACK
    passRepo.saveAll(Pass.generer(sel, s.nbPass))
  }
  return resa // → COMMIT
}

@Modifying @Query("UPDATE Festival f SET f.nbPassVendus = f.nbPassVendus + :n
  WHERE f.idFestival = :id AND f.nbPassVendus + :n <= f.quotaPass")
fun reserverPass(id: Long, n: Int): Int // nb de lignes modifiées
```

GESTION DES TRANSACTIONS

Les propriétés ACID et ce qu'elles apportent

Atomicité

Pass et nuitées réservés ensemble, sinon ROLLBACK complet (scénario 3 du sujet).

Avantage : jamais de Pass sans hébergement, aucun nettoyage à faire après un échec.

Cohérence

Contraintes en base : clé de Nuitee, quota de Pass, CHECK sur notes et dates.

Avantage : même un bug dans le code ne peut pas créer une double réservation.

Isolation

Verrou de ligne Oracle + UPDATE conditionnel : sur la même nuitée, un seul gagne.

Avantage : READ COMMITTED suffit, pas besoin de SERIALIZABLE, plus coûteux.

Durabilité

Le COMMIT Oracle : une réservation confirmée n'est jamais perdue.

Avantage : les PDF ne sont générés qu'après le COMMIT, donc toujours pour une vraie réservation.

POURQUOI CE CHOIX

Quatre façons de réserver une place

Approche	Survente ?	Sous forte charge
SELECT puis UPDATE	**Oui**	Deux clients lisent « libre » en même temps
SELECT … FOR UPDATE	Non	Verrou tenu sur deux allers-retours : file d’attente plus longue
@Version (optimiste JPA)	Non	Échecs en série sur Festival alors qu’il reste des places
UPDATE conditionnel (notre choix)	**Non**	**Un aller-retour, verrou le plus court, échec seulement s’il ne reste rien**

Exemple : 50 demandes simultanées d’1 Pass, 100 places restantes. Avec @Version, beaucoup échouent et réessaient ; avec l’UPDATE conditionnel, **les 50 réussissent**, l’une après l’autre.

GESTION DES TRANSACTIONS

Deux festivaliers, la dernière nuitée

Temps	Festivalier A	Festivalier B
t1	UPDATE … IS NULL → 1 ligne, verrou posé	
t2		UPDATE même nuitée → attend le verrou
t3	**COMMIT : réservation créée**	
t4		Oracle réévalue : idSelection n’est plus NULL → 0 ligne
t5		**ROLLBACK : « validation échouée »**

Résultat : **pas de survente**, et B n’a rien payé pour une place qu’il n’a pas.

PASSAGE À L'ÉCHELLE

Six règles pour tenir la montée en charge

1. Transaction courte

Aucun appel externe dedans. PDF et emails après le COMMIT (@TransactionalEventListener).

2. Point chaud en dernier

L'UPDATE de Festival juste avant le COMMIT : la ligne la plus disputée est verrouillée le moins longtemps.

3. Ordre fixe

Festivals par id, nuitées par (hébergement, date) : pas d'interblocage.

4. Verrous de ligne

Deux hébergements différents ne se bloquent jamais. Index sur lieu, dates et capacité.

5. Pool borné

HikariCP limite les connexions et timeout = 5 s : Oracle n'est jamais submergé.

6. Réessai ciblé

@Retryable sur interblocage ou timeout seulement, jamais sur « plus de place ».

VALIDATION

Comment nous allons le démontrer

Test concurrent

N validations simultanées sur la dernière place ou la dernière nuitée.

Attendu : une seule réussit.

Deux sessions SQL

Démonstration en direct : la 2e session attend le verrou.

Attendu : 0 ligne après le COMMIT de la 1re.

Test de charge

Montée progressive du nombre d’utilisateurs sur un festival populaire.

Attendu : temps de réponse stable, zéro survente.

POINTS À VALIDER AVEC VOUS

Nos questions ouvertes

Tout ou rien sur le panier entier, ou une variante par sélection (SAVEPOINT) ?

nbPassVendus : ajouter ce compteur à Festival plutôt que compter les Pass ?

Hébergements : chambres numérotées (nos nuitées) ou paquets avec un stock ?

Validation anonyme (email + numéro de réservation) : à ajouter ?

Merci, des questions ?